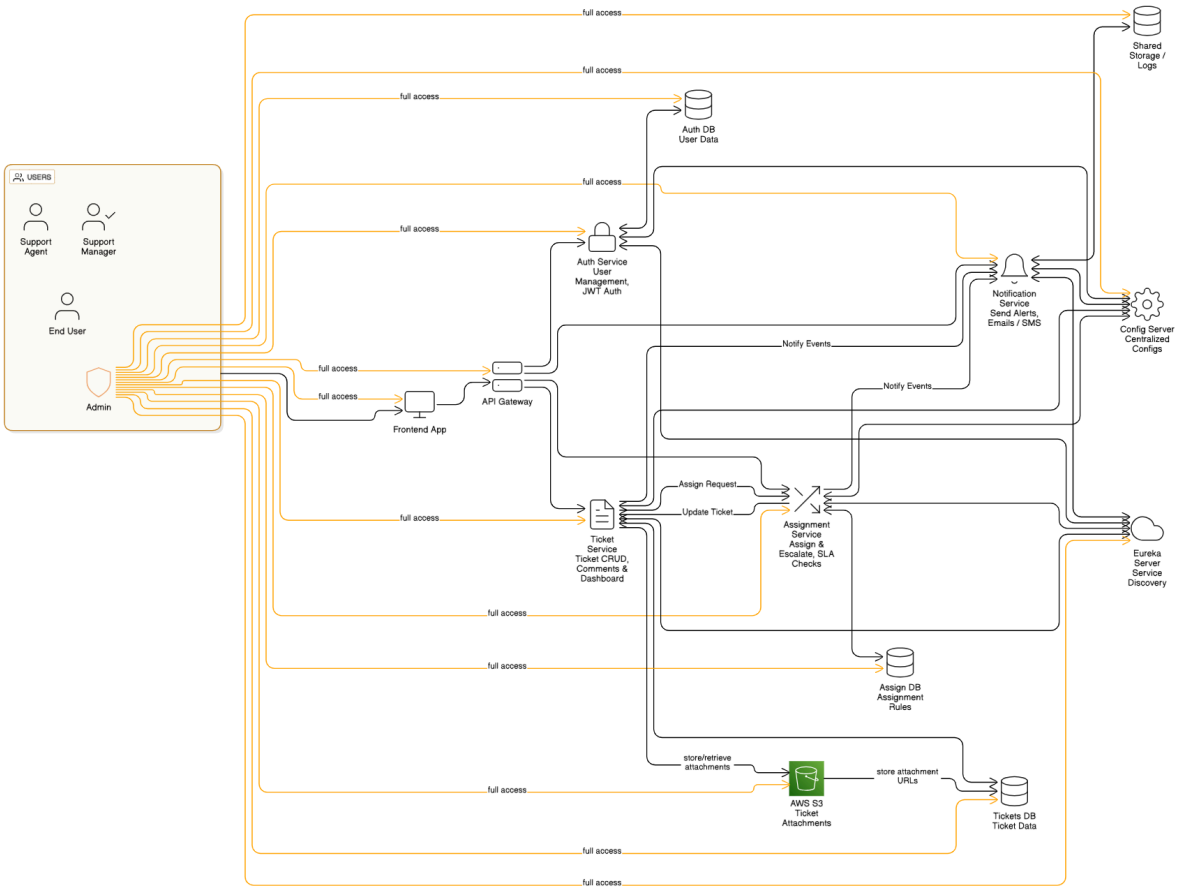
SOFTWARE DESIGN DOCUMENT(SDD)

Smart Ticket & Issue Management System



1. DOCUMENT CONTROL

Item	Details
Project Name	Smart Ticket & Issue Management System
Version	1.0
Author	Sujay Nimmagadda
Date	29/12/2025
Status	Final

2. PURPOSE OF THE DOCUMENT

This document describes the **system architecture, design decisions, component structure, APIs, data models, and non-functional aspects** of the Smart Ticket & Issue Management System.

It is intended for:

- Developers
- Reviewers
- Interview discussions
- Maintenance & enhancement planning

---

3. SYSTEM OVERVIEW

Business Objective

Provide a scalable, secure, and maintainable system to:

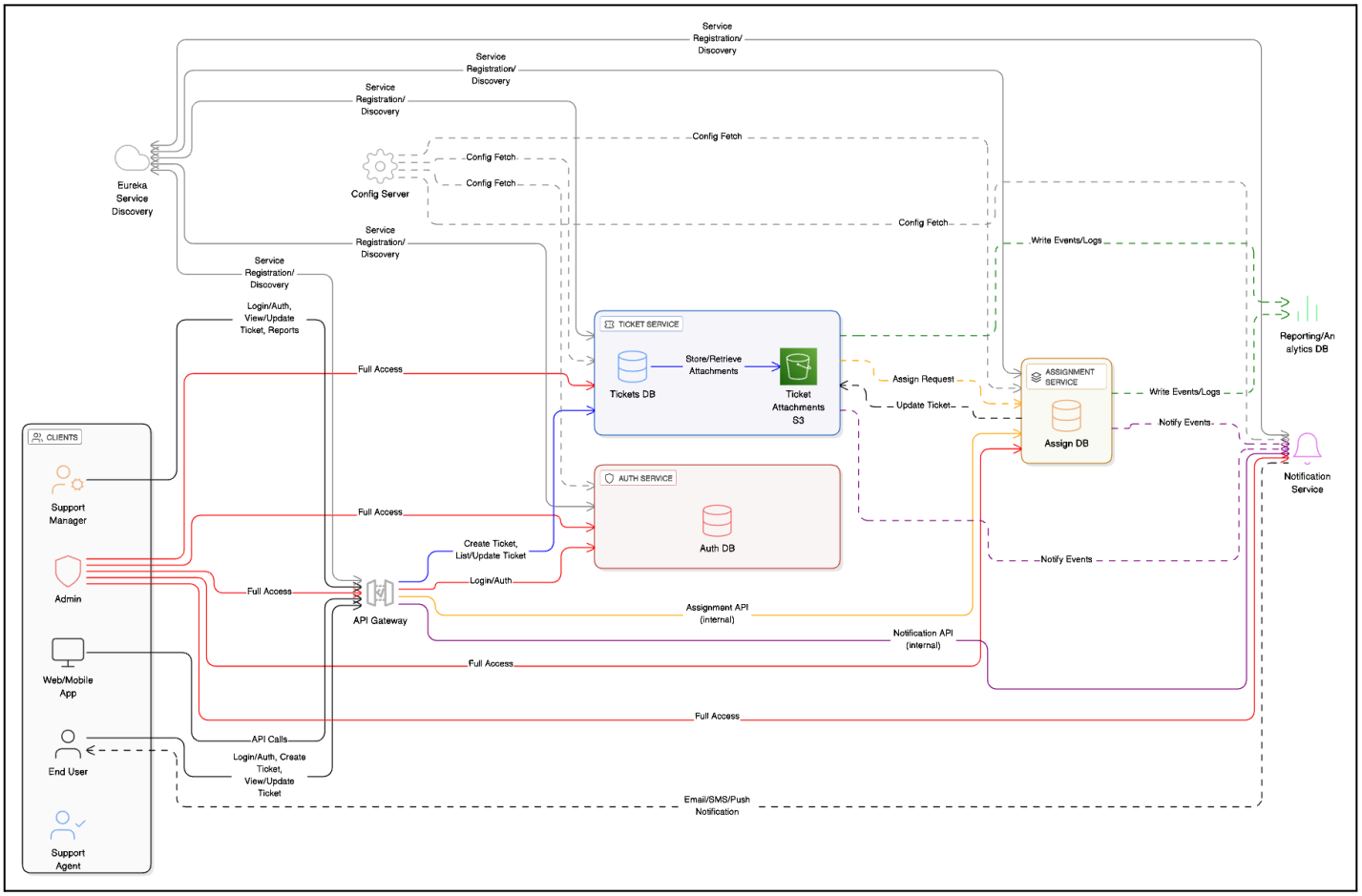
- Manage Users & Identities
- Handle Ticket Lifecycle
- Ticket Allocation
- Support growth through microservices

High-Level Features

- User Registration & Secure Login
- Real-time Ticket Tracking
- Role-based access (USER / ADMIN / Manager / Agent)

---

4. ARCHITECTURE OVERVIEW (HLD)



5. TECHNOLOGY STACK

Backend

- Java 17
- Spring Boot
- Spring Web
- Spring Data MongoDB
- Spring Validation

Frontend

- Angular
- Angular Material
- RxJS

Database

- MongoDB & Postgres

DevOps

- Docker
- Jenkins
- SonarQube

Testing

- JUnit 5
- Mockito
- Spring Boot Test

---

6. MICROSERVICES DESIGN

6.1 Auth Service

Responsibilities

- User registration and login
- JWT token generation and validation
- Role-based access control (RBAC)
- Password encryption (BCrypt)
- User session management
- OAuth2 support

Key Entities

User

├── userId (UUID/Long)

├── username (String)

├── email (String)

├── password (Hashed)

├── role (ADMIN, SUPPORT_MANAGER, SUPPORT_AGENT, END_USER)

├── status (ACTIVE, INACTIVE)

├── createdAt (Timestamp)

├── updatedAt (Timestamp)

└── lastLogin (Timestamp)

Role

├── roleId

├── roleName

├── permissions (Set)

└── description

APIs

Method	Endpoint	Description	Request Body

POST	/auth/login	User login	{
			"username":"string",
			"password": "string"
			
			}

POST	/auth/register	Register	{
			"username":"string",
			"email": "string",
			"password":"string",
			"firstName":"string",
			"lastName": "string"
			
			}

GET	/auth/validate	Validate JWT token	

GET	/user/{userId}	Get user details by Id	{
			"username":"string",
			"email": "string",
			"password":"string",
			"firstName":"string,
			"lastName": "string"
			
			}

PUT	/auth/user/{userId}	Update user profile	{
			
			"useId":"string",
			"email": "string",
			"firstName":"string,
			"lastName": "string"
			
			}

PUT	/user/{userId}	Deactivate user	{
			"userId”
			
			}

PUT	/user/{userId}	Activate user	{
			"userId”
			
			}

POST	/auth/logout	Logout user	
GET	/auth/validate	Validate JWT token	
GET	/auth/health	Health check	-
GET	/users/agents	Get All agents	
GET	/users	Get All users	

GET	/users/managers	Get all Managers	

Database

- ticket_auth_db (postgres)

6.2 Ticket Service

Responsibilities

- Ticket creation and lifecycle management
- Ticket status transitions
- Priority management
- Category management
- SLA tracking
- Ticket search and filtering
- Comments and activity tracking

Key Entities:

Ticket (MongoDB)

├── ticketId (ObjectId)

├── ticketNumber (Unique String)

├── title (String)

├── description (String)

├── category (String)

├── priority (CRITICAL, HIGH, MEDIUM, LOW)

├── status (OPEN, ASSIGNED, IN_PROGRESS, RESOLVED, CLOSED)

├── createdBy (String - User ID)

├── createdAt (Date)

├── updatedAt (Date)

├── assignedTo (String - Agent ID)

├── assignedAt (Date)

├── resolvedAt (Date)

├── closedAt (Date)

├── comments (Array)

│ ├── commentId

│ ├── author

│ ├── content

│ └── timestamp

├── attachments (Array - File URLs)

├── slaStatus (ON_TRACK, BREACHED)

├── responseDueAt (Date)

├── resolutionDueAt (Date)

└── customFields (Map)

Category

├── categoryId

├── name (String)

├── description

└── slaHours (Integer)

Priority

├── priorityId

├── level (1-5)

├── name

└── responseTimeHours

APIs

Method	Endpoint	Description	Request Body

POST	/tickets	Create new ticket	{
			"title": "string",
			"description":"string",
			"category":"string",
			"attachments": ["url1", "url2"]
			}

GET	/tickets/{ticketId}	Get ticket by ID	-

PUT	/tickets/{ticketId}	Update ticket	Request:
			{
			"title": "string",
			"description": "string",
			"category": "string",
			“attachements” []
			}

DELETE	/tickets/{ticketId}	Delete ticket	-
GET	/tickets	List tickets	
GET	/tickets/my-tickets	Get user's tickets	

GET	/tickets/{userId}	Get assigned tickets	

PATCH	/tickets/{ticketId}/status	Change ticket status	Request:
			{
			"status": "ASSIGNED|IN_PROGRESS|RESOLVED|CLOSED",
			"comment": "string (optional)"
			}

POST	/tickets/{ticketId}/comments	Add comment	Request:
			{
			"content": "string",
			"isInternal": true
			}

GET	/tickets/{ticketId}/comments	Get comments	

POST	/tickets/{ticketId}/escalate	Escalate a ticket	Request:
			{
			"userId": "string",
			“userName” string,
			“reason”: string
			}

GET	/ticket/escalated	Get all escalated tickets	

GET	/tickets/{ticketId}/attachments	Get attachments	
GET	/tickets/{ticketId}/activity	Get ticket activity log	

Database

- Ticket collection (MongoDB)

6.3 Assignment Service

Responsibilities

- Manual ticket assignment to agents
- Automatic assignment (round-robin, load-based)
- Assignment history tracking
- Escalation handling
- Skill-based routing
- Workload balancing
- Reassignment management

APIs

Method	Endpoint	Description	Request Body

POST	/assignment/manual	Assign ticket to agent	Request:
			{
			"ticketId":"string",
			"agentId": "string",
			"reason": "string (optional)"
			}

GET	/assign/{assignmentId}	Get assignment details	-

DELETE	/assign/{assignmentId}	Unassign ticket	-
GET	/assignments/agents/available	Get agents with workloads	
GET	/assignments/tickets/unassigned	Get unassigned tickets	

PUT	/assignments/reassign	Reassign a ticket	Request:
			{
			"newAgentId": "string",
			"reason": "Agent unavailable"
			}

GET	/agents/{agentId}/tickets	Get tickets assigned to agent	
GET	/sla/breaches	Get SLA breaches	
GET	/sla/tickets/{ticketId}	Get sla tracking of a ticket	
GET	/sla/active/	Get Active Sla’s	
GET	/sla/warnings	Get sla warnings	

GET	/agents/stats	Display stats for the agent	-

Database

- assignment collection (Postgres)

6.4 Notification Service

Responsibilities

- Email notifications
- Event publishing and consumption
- Status change notifications
- Escalation alerts
- User notifications
- Message queuing via RabbitMQ

Key Entities:

Notification
├── notificationId (UUID)
├── recipientId (String)
├── recipientEmail (String)
├── eventType (TICKET_CREATED, ASSIGNED, STATUS_CHANGED, ESCALATED)
├── subject (String)
├── body (String)
├── createdAt (Timestamp)
├── sentAt (Timestamp)
├── status (PENDING, SENT, FAILED)
└── retryCount (Integer)

Event
├── eventId
├── eventType
├── sourceService
├── sourceEntity (ticketId, userId, etc.)
├── payload (JSON)
└── timestamp

Database

- notification collection (Postgres)

6.5 Api Gateway

Responsibilities

- - JWT Validation
- - Rate Limiting
- - Request/Response Logging
- - CORS Configuration
- - Load Balancer

Routes:

- /auth/** → AUTH-SERVICE (8081)

- /tickets/** → TICKET-SERVICE (8082)

- /assignments/** → ASSIGNMENT-SERVICE (8083)

- /notifications/** → NOTIFICATION-SERVICE (8084)

- /admin/** → API-GATEWAY (Admin endpoints)

6.6 Eureka Server

Responsibilities:

- **Service configuration**

6.7 Config Server

Responsibilities:

- **Centralized configuration**
- **Git repository for storing property files**
- **Supports multiple profiles**

Admin Endpoints

Method	Endpoint	Description	Request Body

PUT	/admin/users/{userId}/role	Update a specific user's system role.	Request:
			{
			"userId":"string",
			“role”: “string”
			}

PUT	/admin/users/{userId}/deactivate	deactivate user	Request: { "userId":"string" }
DELETE	/assign/{assignmentId}	Unassign ticket	-
GET	/assignments/agents/available	Get agents with workloads	
PUT	/admin/users/{userId}/activate	activate user	Request: { "userId":"string" }
PUT	/admin/users/{userId}reset-password	reset password	Request: { "userId":"string" “password” string

			}
GET	/admin/users/stats	Get admin stats	

PUT	/admin/{ticketId}/priority	change priority of ticket	Request:
			{
			"userId":"string"
			“priority”: “string”
			}

PUT	/admin/tickets/{ticketId}/status	change status of ticket	Request:
			{
			"userId":"string"
			“status”: “string”
			}

PUT	/admin/tickets/{ticketId}/category	change category of ticket	Request:
			{
			"userId":"string"
			“category”: “string”
			}

DELETE	/admin/tickets/{ticketId}	delete a ticket	
GET	/admin/tickets/unassigned	Get unassigned tickets	

7. DATA DESIGN (LLD)

User Document

```
{
 "id": "u101",
 "email": "user@test.com",
 "password": "encrypted",
 "role": "USER",
 "active": true
}
```

Ticket Document

```
{
 "id": "p201",
 "title": "Phone",
 "description":"",
 "category":"",
 "priority":"",
 "status": "",
 "created_at": "",
 "updated_at":"",
 "due_date":"",
 "user_id":"",
 "assigned_agent_id"
}
```

Assignment Document

```
{
 "assignmentId": "assgn_8821",
 "ticketId": "T-1000",
 “agentId”:”agent_1”,
 “assignedBy”:”manager_1”,
 “assignedAt”:”Time Stamp”,
 “priority”:”HIGH”,
 “sla”:{
   “responseTimeLimit”:”Time Stamp”,
   “resolutionTimeLimit”:”TIme STamp”,
   “isEscalated”:boolean
 },
 “status”:”ACTIVE/INACTIVE”
}
```

Notification Document

```
{
 "id": "N1001",
 "emailId": "abc@gmail.com",
 "description":"",
 "created_at": "Time Stamp",
}
```

---

8. API DESIGN & VALIDATION

- RESTful principles
- JSON request/response
- Bean Validation at DTO layer
- Service-level business rule enforcement
- Standard HTTP status codes

---

9. ERROR HANDLING STRATEGY

Global Exception Handling

- Centralized using @ControllerAdvice
- Standard error response format

```
{
 "timestamp": "TimeStamp",
 "status": 400,
 "error": "Validation Error",
 "message": "Invalid Ticket Data"
}
```

---

10. SECURITY DESIGN

- Password encryption (BCrypt)
- Role-based access control
- JWT authentication
- Secure API access

---

11. NON-FUNCTIONAL REQUIREMENTS

Area	Design Decision
Scalability	Stateless services, Docker
Performance	Pagination, async calls
Availability	Independent services
Maintainability	POM-like layered backend
Security	Validation, encryption
Observability	Logging & monitoring

---

12. TESTING STRATEGY

Backend

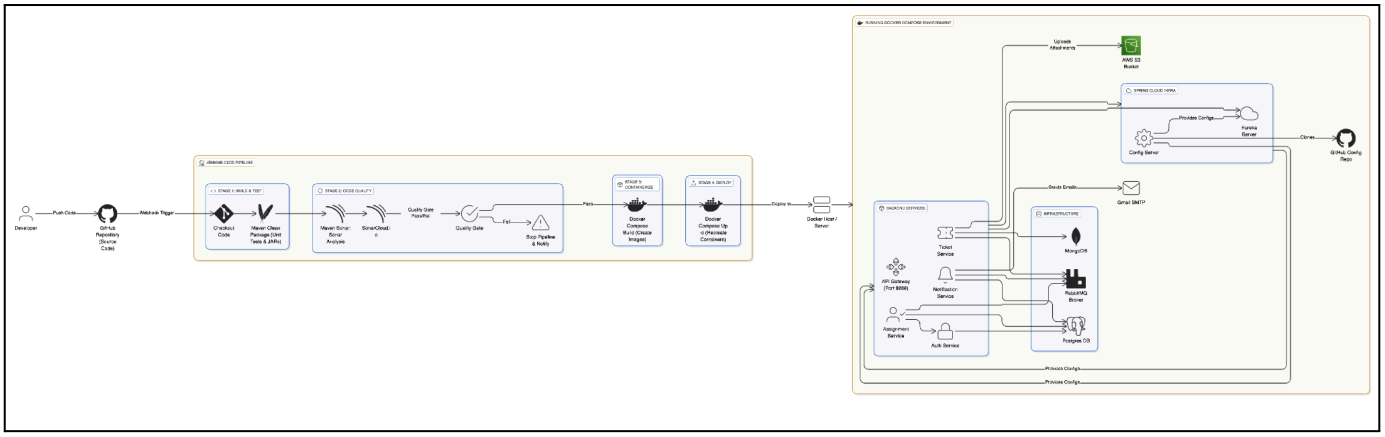
- Unit tests (Service layer, controller layer)
- Controller tests (MockMvc)

Quality Gates

- SonarQube enforced
- Build fails on violations

---

13. CI/CD DESIGN



Pipeline Flow

1. Git Commit
2. Jenkins Build
3. Unit Tests
4. SonarQube Scan
5. Quality Gate Check
6. Docker Build
7. Docker Compose Deploy

---

15. ASSUMPTIONS & CONSTRAINTS

Assumptions

- **Infrastructure Availability**: It is assumed that the hosting environment (Docker Engine) provides sufficient CPU and RAM (minimum 8GB) to run 7+ concurrent Java microservices and 3 database instances.

- **External Service Uptime**: Third-party services, including **AWS S3** for storage and **Gmail SMTP** for notifications, are assumed to have a 99.9% uptime.

- **Continuous Availability of CI/CD**: The Jenkins server and SonarCloud are assumed to be accessible whenever a code push occurs to the GitHub repository.

Constraints

- **Data Storage**: Persistent data must be split between **PostgreSQL** (relational/identity data) and **MongoDB** (unstructured ticket metadata).

- **Security Protocol**: All microservice communication and end-user access must be secured using **JWT (JSON Web Tokens)** and role-based access control (RBAC).

- **Asynchronous Messaging**: All event-driven communication (e.g., ticket assignments, notification triggers) must utilize **RabbitMQ** as the message broker.

---

16. FUTURE ENHANCEMENTS

- **Sentiment Analysis**: Implement NLP models to analyze the tone of incoming tickets. High-priority alerts can be triggered if a customer's sentiment is detected as "frustrated" or "urgent."

- **AI Smart Routing**: Replace rule-based assignment with machine learning models that route tickets based on an agent's historical resolution speed and success rate for specific issue categories.

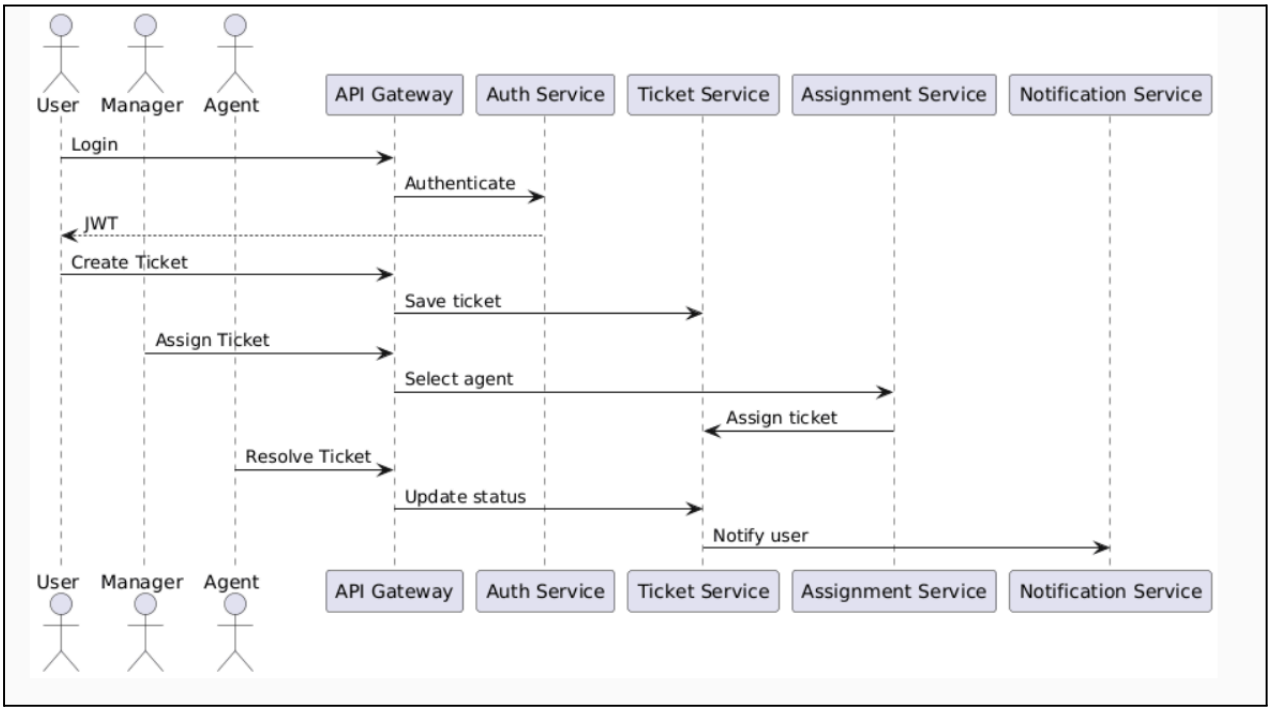
- **Auto-Summary**: Use LLMs (Large Language Models) to generate a concise summary of long ticket threads for managers during shifts or escalations.

---

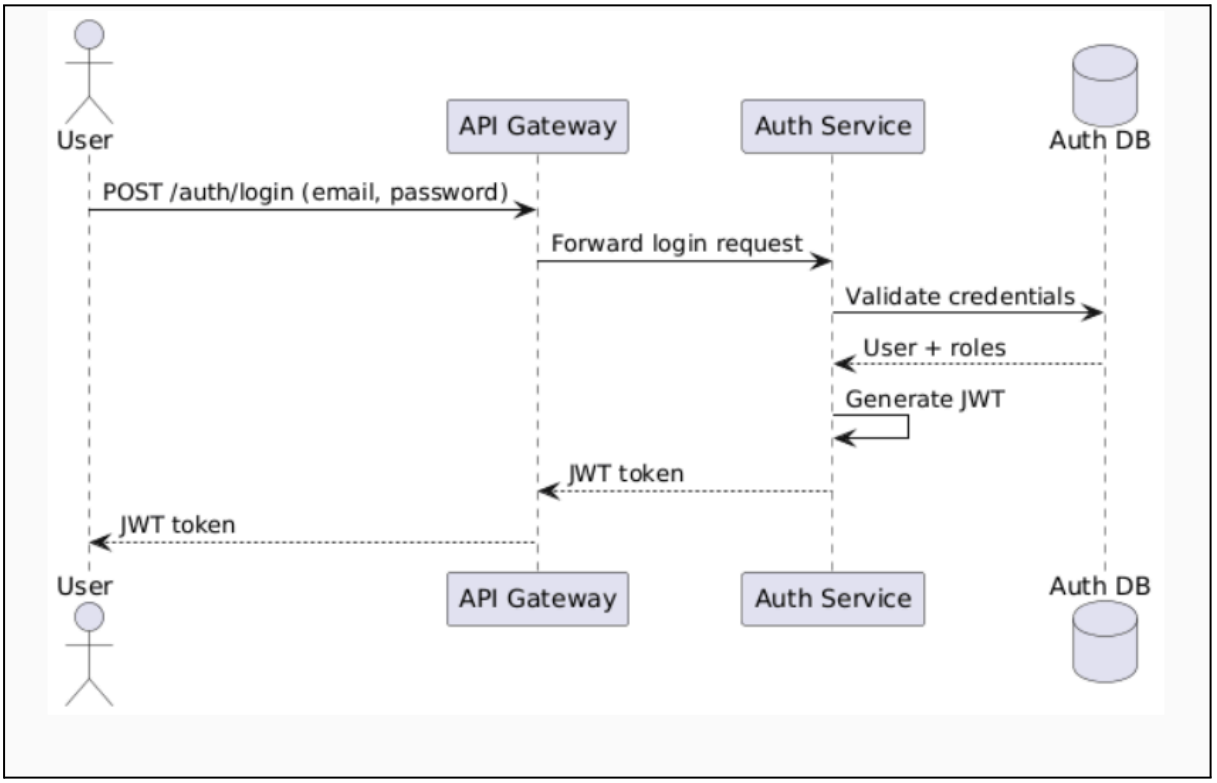
17. CONCLUSION

This design ensures:
✔ Clean separation of concerns
✔ Scalability & maintainability
✔ Testability & CI/CD readiness

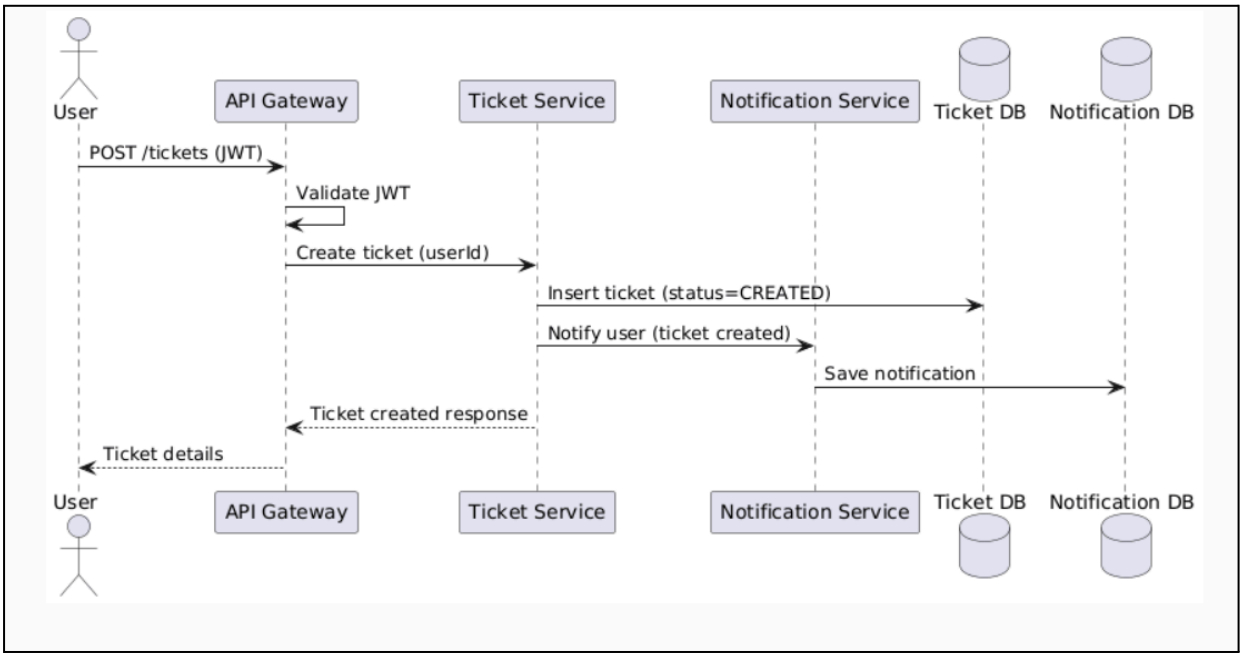
SEQUENCE DIAGRAMS



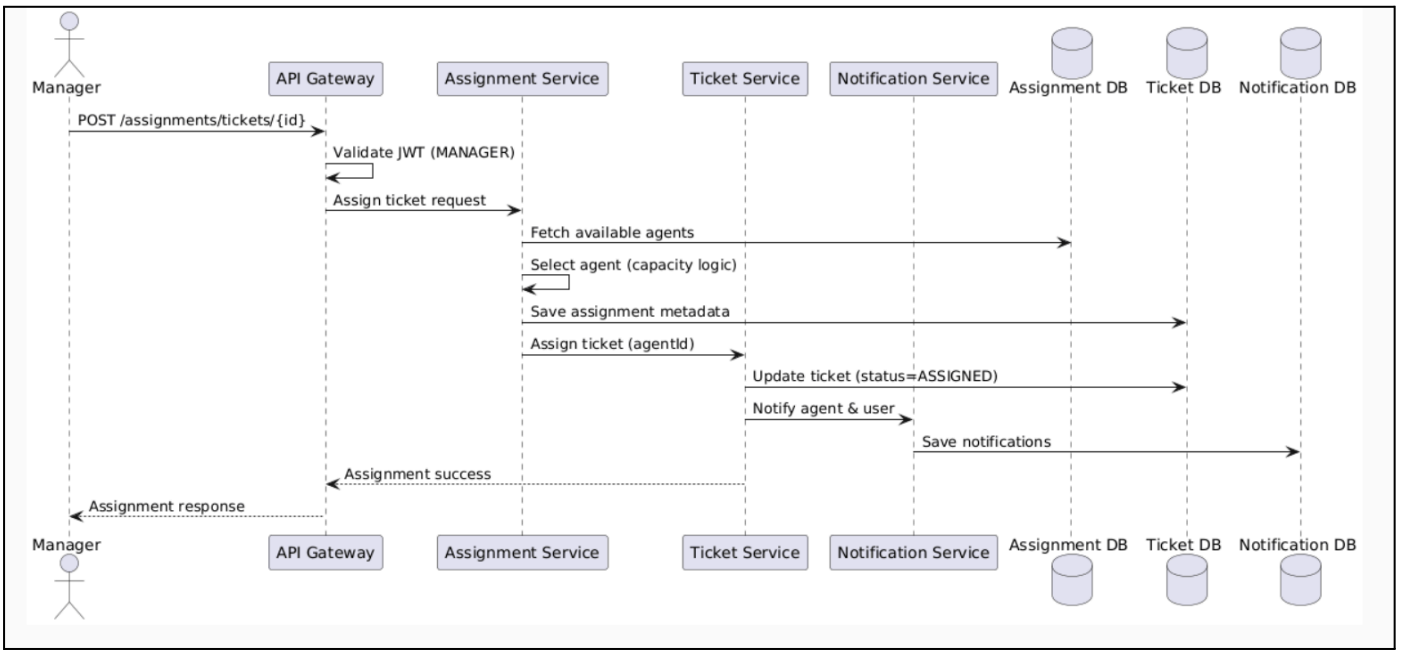
User Login Flow -



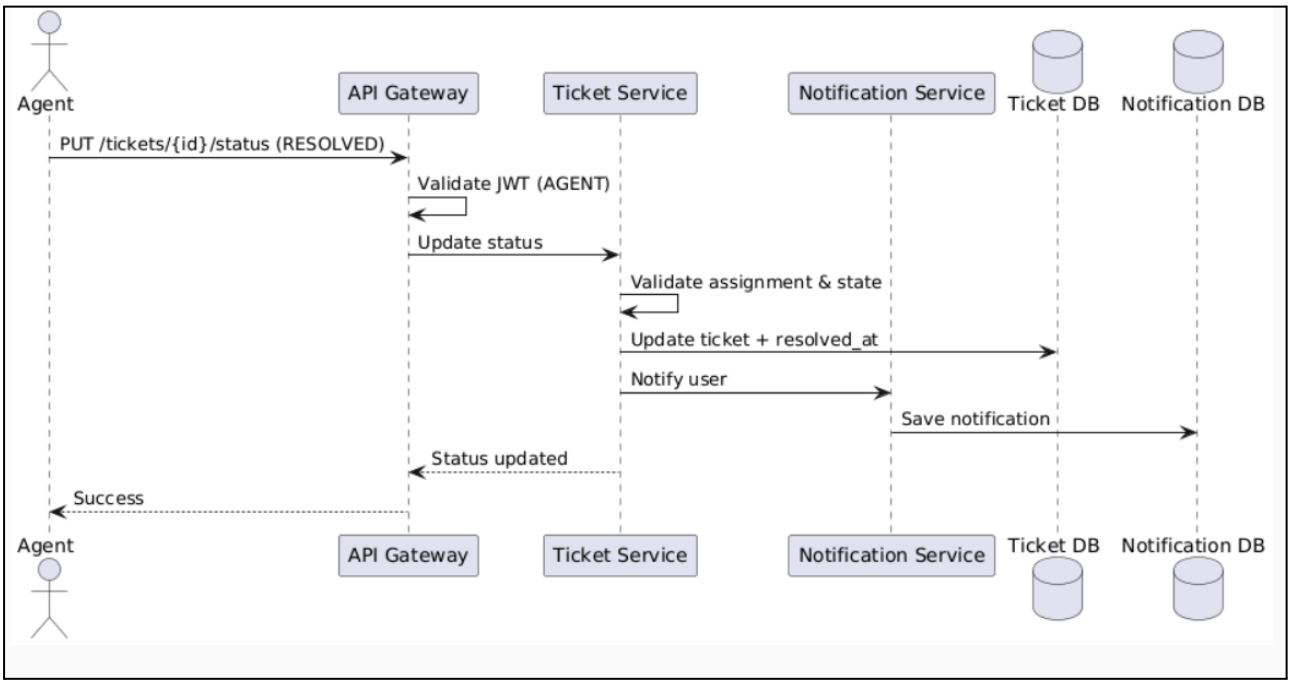
Ticket Creation -



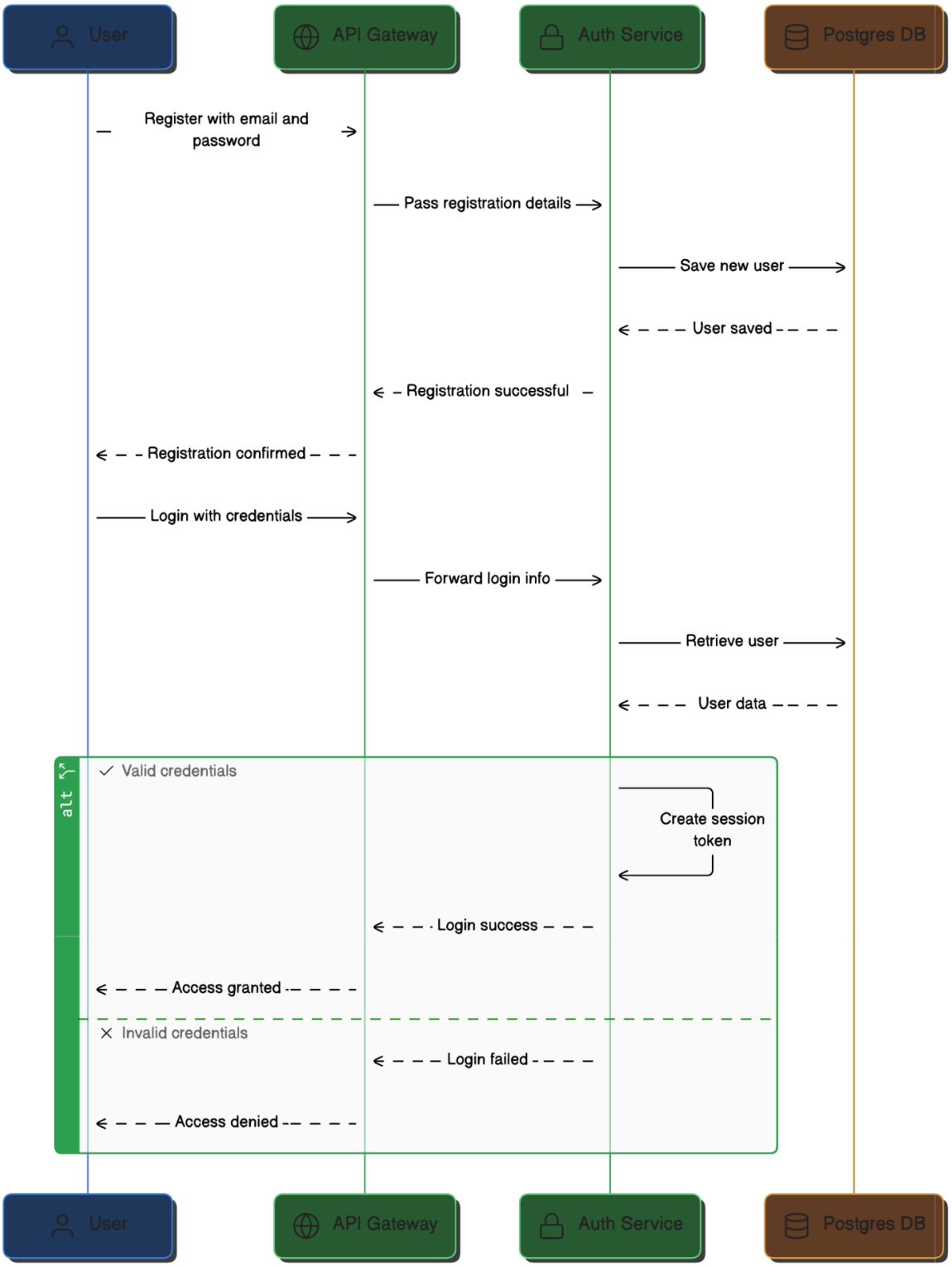
Manager Assigns Ticket to Agent -



Agent Resolves Ticket -



2. User Login — Sequence Diagram



Scenario

Registered user logs into the system.

Flow

1. User submits login form
2. Angular calls POST /users/login
3. User Service fetches user from Postgres
4. Password is verified
5. JWT token generated
6. Response sent to Angular
7. Angular stores token and navigates user

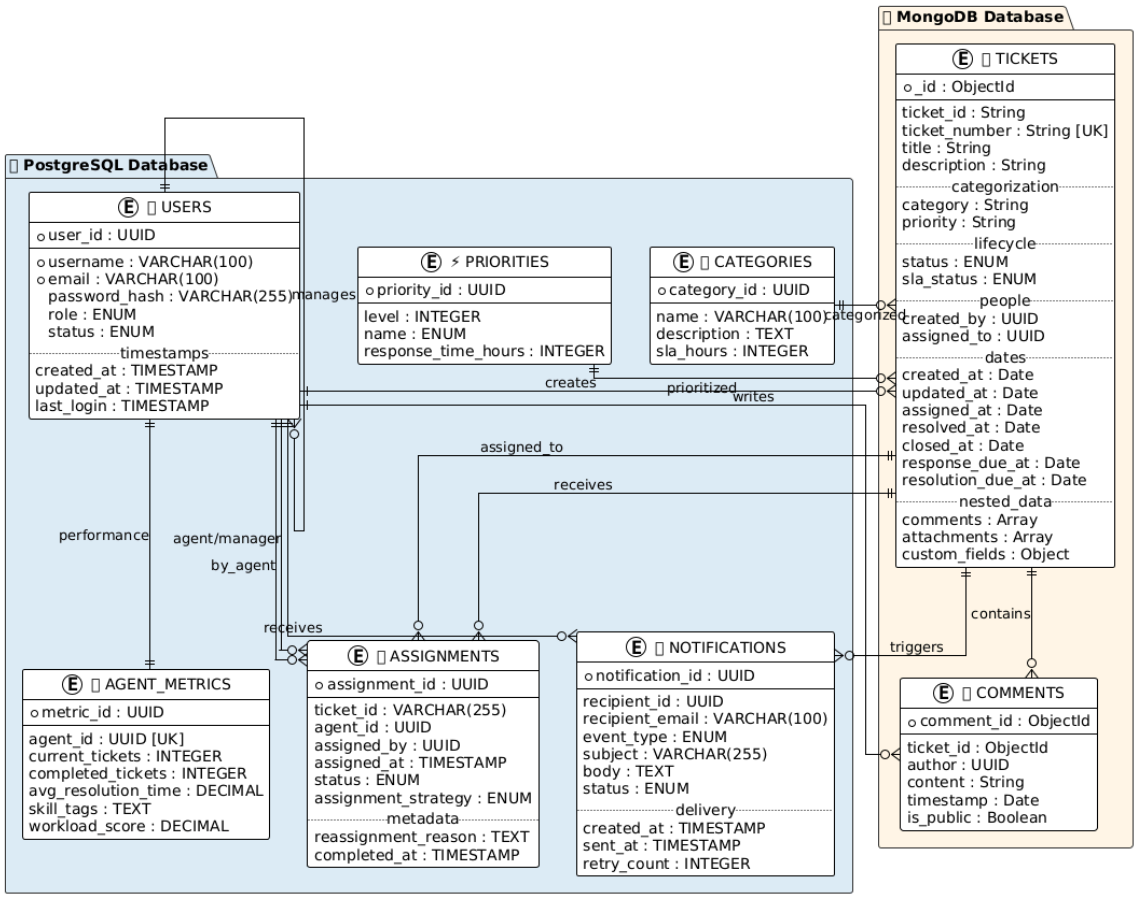
Participants

- User
- Angular UI
- User Service
- MongoDB

Failure Cases

- Invalid credentials → 401
- Inactive user → 403

ER Diagram



Business Rules check them clearly

1.1 Ticket Creation Rules

Rule BR-001: Public User Can Create Tickets

- User must have END_USER role
- User must be active (is_active = true)
- Title: 5-200 characters
- Description: 10-5000 characters
- Must select valid category & priority

Rule BR-002: Auto-Assign Based on Queue

- Ticket auto-assigned to least-loaded agent with that category expertise
- Load = number of open tickets per agent
- If no agent available, mark as unassigned

Rule BR-003: Ticket Numbering

- Format: TKT-YYYYMMDD-00001
- Example: TKT-20251229-00001
- Sequential per day

Rule BR-005: Valid Status Transitions

- OPEN → IN_PROGRESS, REOPENED, CLOSED

IN_PROGRESS → RESOLVED, REOPENED, CLOSED

RESOLVED → CLOSED, REOPENED, IN_PROGRESS

CLOSED → REOPENED (within 7 days)

REOPENED → IN_PROGRESS, CLOSED

Status Definitions:

- OPEN: Newly created, awaiting assignment

- IN_PROGRESS: Agent is working on it

- RESOLVED: Agent has fixed the issue

- CLOSED: Customer confirmed resolution

- REOPENED: Customer confirmed issue still exists

Rule BR-006: Status Change Notifications

- OPEN → IN_PROGRESS: Notify assigned agent
- IN_PROGRESS → RESOLVED: Notify ticket creator
- RESOLVED → CLOSED: Notify ticket creator
- CLOSED → REOPENED: Notify assigned agent & manager

Rule BR-007: Mandatory Comments on Status Change

- When changing to RESOLVED or CLOSED, agent MUST provide explanation comment
- If comment is empty, validation error thrown

1.3 Assignment Rules

Rule BR-008: Assignment Restrictions

- Cannot assign to END_USER (only AGENT/MANAGER can be assigned)
- Cannot assign to inactive user
- Cannot assign already-assigned ticket without unassigning first
- Only MANAGER or ADMIN can assign tickets

Rule BR-009: Reassignment Tracking

- Notify manager about high reassignment count
- Purpose: Identify problematic tickets

Rule BR-010: Agent Queue Management

- When ticket assigned, add to agent's queue
- Queue position based on priority (CRITICAL first, then HIGH, MEDIUM, LOW)
- CRITICAL tickets bypass normal queue

1.4 Comment & Attachment Rules

Rule BR-011: Internal Comments

- Internal comments only visible to support staff (AGENT, MANAGER, ADMIN)
- Customers cannot see internal comments
- Mark with isInternal = true flag

Rule BR-012: Comment Editing/Deletion

- Only comment author or ADMIN can edit
- Can only edit within 24 hours of creation
- Maintain edit history
- Delete only with ADMIN permission

Rule BR-013: Attachment Constraints

- Maximum 50 MB per attachment
- Maximum 10 attachments per ticket
- Allowed types: PDF, Image (JPG, PNG), Word, Excel, Text
- Blocked types: .exe, .bat, .sh, .jar, .dll

1.5 Customer Satisfaction Rules

Rule BR-014: Post-Resolution Survey

- Trigger: Ticket status changes to CLOSED
- Action: Send survey email after 24 hours
- Question: "Were we able to resolve your issue?"
- Options: Very Satisfied, Satisfied, Neutral, Unsatisfied, Very Unsatisfied

Rule BR-015: Re-open Policy

- Customer can reopen within 7 days of closure
- After 7 days, must create new ticket

- Auto-assign to previous agent if available
- Previous agent notified of reopening

2. SLA RULES (Service Level Agreements)

Priority	Response Time	Resolution Time	Escalation
CRITICAL	15 minutes	2 hours	30 minutes
HIGH	1 hour	8 hours	2 hours
MEDIUM	4 hours	24 hours	8 hours
LOW	12 hours	48 hours	24 hours

5. ESCALATION RULES

Rule ESC-001: Manual Escalation by Agent

- When: Agent cannot resolve and needs help
- Who: SUPPORT_AGENT or SUPPORT_MANAGER
- Target: Next level manager
- Reason: Required (free text)
- Notification: Immediate to target manager

Rule ESC-002: Auto-Escalation on SLA Breach

- When: SLA escalation time exceeded
- Who: System (automatic)
- Trigger: now > (createdAt + escalationAfterMinutes)
- Target: Escalation path from SLA rule